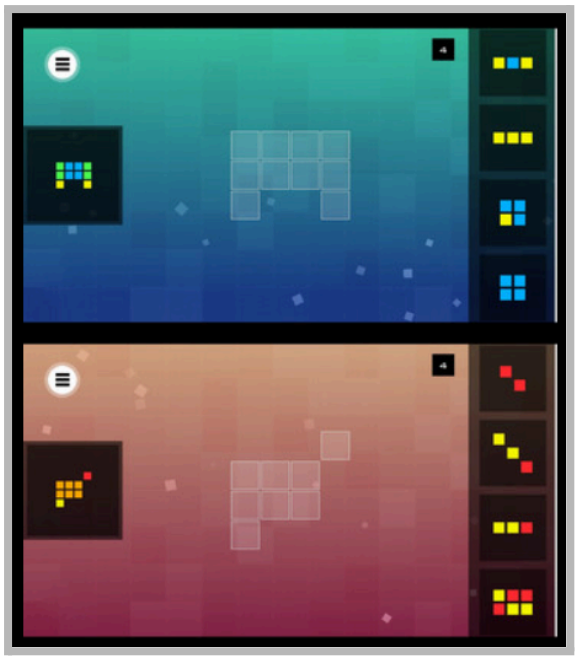
Penerapan Algoritma Pencarian A* untuk Menyelesaikan Puzzle Color Mind

Naftali Aditya Agung Dwi Nugraha - 232300333
Program Studi IT and Big Data Analysis
Sekolah Sains dan Teknik
Calvin Institute of Technology. Jakarta 10610, Jakarta Pusat
nnugraha33@students.calvin.ac.id

Abstrak

I. Pendahuluan



Gambar 1.1 Permainan Color Mind
https://lyl-games.itch.io/color-mind

Color Mind adalah permainan puzzle Indie yang diproduksi oleh LYL Games. Game Berdasarkan deskripsi game ini, game ini menawarkan permainan yang meliputi logika untuk mengasah pikiran dengan ratusan level yang telah disiapkan untuk menantang otak kita untuk berlogika. Tujuan dari game ini ialah menyusun *puzzle* dengan menggunakan balok-balok yang telah disediakan untuk mencapai sesuai target yang telah disediakan pada tiap level. Uniknya, tiap balok memiliki warna pada tiap kotak-kotaknya, dan warna-warna ini dapat dicampur dengan warna primer lain untuk menghasilkan warna lain. Tugas pemain adalah untuk memikirkan cara bagaimana untuk menyusun balok-balok tersebut sehingga melalui penggabungan warna, maupun pemutaran balok, ataupun hanya sekedar meletakkan pada slot tersedia dapat mencapai sesuai dengan target tiap level. Permainan ini menarik karena memerlukan kemampuan berpikir dampak dari setiap pilihannya, dikarenakan tempat yang terbatas dan pilihan balok yang terbatas, sehingga membutuhkan kemampuan berpikir kreatif dan juga sistematis untuk menyelesaikan permainan ini.

Salah satu cara berpikir untuk menyelesaikan teka-teki ini adalah teknik penyelesaian algoritma *Shortest Path.* Algoritma *Shortest Path* berusaha menyelesaikan masalah klasik dalam optimisasi kombinatorial *Shortest Path Problem* [1] . Algoritma *Shortest Path* pada dasarnya bertujuan untuk mencari rute terpendek yang menghubungkan dua titik. Cara menyelesaikan teka-teki ini dengan pendekatan tersebut adalah dengan melihat bahwa pada tiap kondisi, kita diberi beberapa pilihan sebagai langkah berikutnya kita dengan tujuan untuk semakin lama semakin mendekati solusi dari level tersebut. Tiap pemilihan keputusan dapat dilihat sebagai dua titik yang terhubung, titik mula dan titik setelah kita terapkan keputusan tersebut. Kita dapat menerapkan pendekatan algoritma *shortest path* untuk mencari jalur mana

yang pada akhirnya menjadi sesuai dengan target level tersebut.

Salah satu algoritma pencarian *shortest path* adalah algoritma A*. Algoritma pencarian A*, atau dapat disebut sebagai algoritma pencarian *A-star*, dipilih karena adalah algoritma pencarian *shortest path* terbaik, yang dapat menyelesaikan pencarian dengan waktu sangat cepat walaupun tidak selalu optimal [2].

Dalam makalah ini, kita akan membahas bagaimana menerapkan algoritma pencarian A* untuk menyelesaikan permainan teka-teki ini. Pembahasan akan mencakup penjelasan teori dasar A*, representasi matriks dari puzzle, dan juga evaluasi dari performa penerapan algoritma A* dalam masalah ini. Makalah ini ditulis dengan tujuan untuk menyelesaikan teka-teki permainan Color Mind secara spesifik dan tidak bertujuan untuk memajukan penelitian algoritma pencarian A* maupun penyelesaian *Shortest path problem*.

II. Dasar Teori

A. Algoritma Pencarian A*

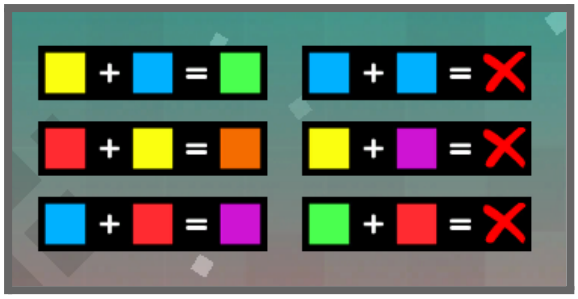
Dalam penyelesaian masalah *shortest path problem*, banyak algoritma-algoritma muncul untuk menyelesaikan untuk masalah itu, beberapa contoh populer ialah BFS, DFS, Dijkstra, dan juga A*. BFS dan DFS merupakan pendekatan pencarian secara menyeluruh tanpa optimisasi, keduanya memiliki keunggulan yaitu kepastian untuk jawaban yang optimal, dikarenakan dilakukan pencarian terhadap semua kemungkinan, namun hal itu juga menyebabkan algoritma tersebut menjadi sangat lambat [4]. Dijkstra menggunakan pendekatan BFS namun menambahkan pendekatan greedy dalam menyelesaikan masalah ini [5]. Sedangkan algoritma pencarian A* adalah pengembangan dari algoritma Dijkstra [6] yang menggunakan "estimasi heuristik" untuk membantu saat proses pencarian jalan, mengorbankan akurasi, optimalitas, kelengkapan, atau presisi demi kecepatan. Sedangkan heuristik adalah estimasi seberapa jauh kita dari sebuah tujuan [3]. Mengutip dari jurnal berjudul “OPTIMIZATION OF A-STAR SEARCH ALGORITHM“ karya Piskorskii, Abdullin, dan Nikolaeva, berikut adalah cara kerja algoritma A*[6]

<u>Langkah-langkah algoritma A*:</u>

1. Siapkan antrian untuk titik yang akan dianalisis, dan catat titik yang belum dianalisis
2. Untuk Semua tetangga dari titik mula, hitung koefisien F_n dengan rumus $F_n = G_n + H_n$, dimana G_n adalah biaya transisi dari titik mula ke titik saat ini, dan H_n merupakan jarak yang tersisa dari titik saat ini sampai ke titik selesai. Masukan semua tetangga tersebut ke dalam antrian beserta dengan nilai F_n masing-masing.
3. Dari antrian, pilih titik dengan nilai F_n terendah untuk dianalisa. Jika titik tersebut masuk ke daftar poligon yang sudah diperiksa, lewati perhitungan. Jika titik tersebut belum dianalisis, maka kita tambahkan pada antrian dan hitung nilai F_n.
5. Ulangi langkah 2 hingga 4 hingga titik akhir rute tercapai
6. Jika semua titik telah dianalisa dan titik akhir masih belum tercapai, maka rute tersebut tidak ada

B. Aturan Permainan

Peraturan penting yang penting untuk diperhatikan ialah peraturan pencampuran warnanya, yaitu hanya ada 3 pencampuran yang sah, sisanya tidak sah sehingga tidak bisa menaruh sebuah balok jika ada satu pun sel yang melanggar peraturan penggabungan warna ini.



Gambar 2.1 Permainan Color Mind
https://lyl-games.itch.io/color-mind

C. Representasi Matrix dari Puzzle

Untuk memodelkan target yang ingin kita capai, tempat kerja kita, dan juga balok-balok yang kita miliki, dalam algoritma ini direpresentasikan sebagai numpy ndarray [7] agar komputer dapat lebih mudah melakukan komputasi karena dalam bentuk yang lebih terstruktur dibandingkan dengan gambar. Setiap warna memiliki kode nya dan pengguna aplikasi perlu memasukkan secara manual sesuai dengan singkatan yang telah disediakan. Setelah user melakukan input, algoritma akan mengubah tiap singkatan berupa huruf alfabet tersebut menjadi representasi numeric ascii dari huruf tersebut dikarenakan ndarray dari numpy hanya dapat menyimpan angka dan tidak dapat menyimpan huruf apapun.

```
Constructing target matrix...
Abbreviations :
Yellow : Y
Blue : B
Red : R
Orange : O
Purple : P
Green : G
Input row 1 of Target's matrix: G B B G
Input row 2 of Target's matrix: G B B G
Input row 3 of Target's matrix: Y . . Y
Input row 4 of Target's matrix:

Target matrix:
[G] [B] [B] [G]
[G] [B] [B] [G]
[Y]         [Y]
```

Gambar 2.1 Representasi Matrix Target
(Sumber : Dokumentasi Pribadi)

III. Implementasi

Saya menggunakan bahasa Python dengan Python versi 3.12.1 untuk projek ini. Beberapa library yang perlu di *import* sebelum menjalankan program ini ialah numpy, os, time, dan juga class Priority Queue dari queue.

1. Class Game

Class Game bertujuan untuk menyimpan fungsi-fungsi untuk menjaga agar peraturan game terus terpenuhi. Class Game meliput dictionary yang menyimpan kombinasi kombinasi yang mungkin untuk dilakukan, dan juga ada dictionary yang menyimpan warna apa saja yang adalah warna dasar dari sebuah warna target.

```
combinations :dict[tuple, int] = {
        (ord("Y"), ord("B")) : ord("G"),
        (ord("R"), ord("Y")) : ord("O"),
        (ord("B"), ord("R")) : ord("P")
    }

    half_points :dict[int, set] = {
        ord("B") : {ord("G"), ord("P")},
        ord("Y") : {ord("G"), ord("O")},
        ord("R") : {ord("O"), ord("P")}
    }
```

Gambar 3.1 Data Dictionary
(Sumber : Dokumentasi Priabdi)

Kemudian ada method-method untuk tahap persiapan game, seperti mendapatkan matriks template untuk mengerjakan level tersebut. Method-method ini hanya menciptakan objek-objek matriks.

```python
@staticmethod
    def create_target() -> Matrix:
        print("\nConstructing target matrix...")
        print("Abbreviations : ")
        print("Yellow : Y \nBlue : B\nRed : R\nOrange : O\nPurple : P\nGreen : G")
        obj = InputMatrix("Target")
        obj.print_matrix()
        return obj

    @staticmethod
    def create_board(target) -> Matrix:
        print("\nAuto constructing board matrix...")
        obj = Board(target)
        obj.print_matrix()
        return obj

    @staticmethod
    def create_blocks() -> list[Block]:
        print("\nConstructing blocks...")
        amount = int(input("How many blocks are available? "))

        print("\nabbreviations : ")
        print("Yellow : Y \nBlue : B\nRed : R\nOrange : O\nPurple : P\nGreen : G")
        blocks :list[Block] = []
        for i in range(amount):
            blocks.extend(Block(f"Block {i+1}").rotations)

        print("Blocks: ")
        for block in blocks:
            block.print_matrix()

        return blocks
```

Gambar 3.2 Method Tahap Persiapan
(Sumber : Dokumentasi Priabdi)

Method *is_feasible* berfungsi untuk memeriksa apakah sebuah balok dapat diletakan dalam kordinat tertentu. Cara memeriksanya ialah dengan cara pencarian lengkah, untuk setiap sel kita periksa apakah sel tersebut dapat diletakan pada sel itu. Jika ada satu peraturan pun dilanggar, method ini langsung mengembalikan nilai False.

```python
@staticmethod
    def is_feasible(y :int, x :int, block : Block, board : Matrix) -> bool:
        for row in range(y, y + block.height):
            for col in range(x, x + block.width):
                if block[row-y][col-x] == ord("."):
                    continue
                elif row >= board.height or col >= board.width:
                    return False
                elif board[row][col] == ord("."):
                    return False
                elif board[row][col] == ord("-"):
                    continue
                elif (board[row][col], block[row-y][col-x]) in Game.combinations\
                      or (block[row-y][col-x],board[row][col] ) in Game.combinations:
                    continue
                else:
                    return False
        return True
```

Gambar 3.3 Method is_feasible
(Sumber : Dokumentasi Priabdi)

Setelah diperiksa kevalidan menaruh sebuah balok dalam koordinat tertentu, gunakan fungsi *put* untuk mensimulasikan menaruh balok tersebut pada koordinat tersebut secara iteratif tiap selnya

.

```python
@staticmethod
    def put(pos_y :int, pos_x :int, block : Block, board:Matrix) -> None:
        for y in range(pos_y, pos_y + block.height):
            for x in range(pos_x, pos_x + block.width):
                if block[y-pos_y][x-pos_x] == ord("."):
                    continue
                elif board[y][x] == ord("-"):
                    board[y][x] = block[y - pos_y][x - pos_x]
                else:
                    board[y][x] = Game.combine(board[y][x], block[y - pos_y][x - pos_x])
```

Gambar 3.5 Method *put*
(Sumber : Dokumentasi Priabdi)

Untuk membantu dalam fungsi heuristik yang dibutuhkan oleh algoritma A*, class *Game* menyediakan method *offset* yang mengembalikan perkiraan jarak dari matriks yang sedang dikerjakan dari target.

```python
@staticmethod
    def offset(board : Matrix, target:Matrix):

        ans = 0
        for row in range(target.height):
            for col in range(target.width):
                if board[row][col] == target[row][col]:
                    ans += 2
                elif board[row][col] in Game.half_points.get(target[row][col],[]):
                    ans += 1

        return (target.height * target.width * 2) - ans
```

Gambar 3.6 Method *offset*
(Sumber : Dokumentasi Priabdi)

Cara perhitungannya ialah untuk setiap warna yang sudah sesuai posisi nya dengan target akan mendapat 2 poin pada nilai akhir, untuk setiap warna yang merupakan warna dasar dari target akan mendapat 1 poin, sedangkan sel yang masih kosong maupun masih salah tidak mendapat poin sama sekali. Optimisasi 1 poin pada warna dasar sangat penting karena dapat membantu keakuratan fungsi heuristik algoritma pencarian A*. Tanpa itu algoritma akan memperlakukan warna-warna yang berpotensi digabungkan menjadi sesuai target sama dengan sel yang kosong, sehingga estimasi fungsi heuristik tidak optimal karena salah dalam memberikan jarak perkiraan menuju tujuan akhir.

2. Class GameSolver

Pada inisiasi objek GameSolver, dilakukan tahap persiapan yaitu

mengumpulkan matriks-matriks penting seperti target matriks sebagai acuan target yang ingin dicapai, board matriks sebagai matriks kosong tempat kita meletakkan balok-balok yang tersedia, dan *list* yang memuat matriks balok-balok untuk mencatat semua balok yang mungkin kita gunakan, termasuk rotasi yang mungkin dari tiap balok.

```python
def __init__(self):
        self.game :Game = Game
        self.target :Matrix = self.game.create_target()
        self.board :Matrix = self.game.create_board(self.target)
        self.blocks :list[Block] = self.game.create_blocks()
        self.solution()
```

Gambar 3.6 Dunder __*init*__
(Sumber : Dokumentasi Priabdi)

Setelah tahap inisiasi telah dijalankan, semua data yang kita perlukan sudah tersedia, kita bisa langsung menjalankan algoritma A* untuk mencari jalur untuk menyelesaikan masalah pada level tersebut

```python
def solve(self) -> list[str]:
        print("Solving...\n")
        start = Node(self.board, list(), list(), self.target)
        queue = PriorityQueue()
        queue.put((start.offset, start))

        trials = 1
        while not queue.empty():
            node :Node = queue.get(0)[1]

            if node.offset == 0:
                return "Success", node.used, node.coord, trials

            for idx in range(len(self.blocks)):
                if idx in node.used:
                    continue

                for y in range(self.target.height):
                    for x in range(self.target.width):
                        if self.game.is_feasible(y, x, self.blocks[idx], node.board ):
                            copy = BuildMatrix("Copy", node.board.width, node.board.height, node.board.matrix)
                            self.game.put(y, x, self.blocks[idx], copy)
                            new_node = Node(copy, node.used + [idx], node.coord + [f"({y},{x})"], self.target)
                            queue.put((new_node.offset, new_node))

            trials += 1

        return "Failled", None, None, trials
```

Pencarian A* pada program ini diimplementasikan dengan langkah-langkah berikut :

1. Buat node/state awal yang merepresentasikan kondisi awal permainan saat papan masih kosong.
2. Selama antrian tidak kosong, ambil node dengan nilai fungsi heuristik paling optimal, yaitu yang paling mendekati target dan block yang telah digunakan paling minimal
3. Periksa keakuratan node tersebut, jika ternyata sudah sesuai dengan target, maka akan langsung return solusi kepada fungsi pemanggilanya.
4. Jika belum sesuai dengan target, eksplorasi tetangga yang memungkinkan, yang dimaksud dengan memungkinkan ialah penaruhan balok di kordinat tertentu sah sesuai peraturan permainan. Untuk membuat node tetangga, perlu dibuat copy dari matriks yang sedang dikerjakan agar tidak ada pointer yang mengarah ke objek yang sama sehingga modifikasi terhadap satu objek berpengaruh pada objek lainnya
5. Jika antrian habis dan belum ditemukan solusi, maka dapat dipastikan tidak ada solusi untuk puzzle tersebut, atau bisa jadi ada kesalahan pada tahap input.

.

Setelah algoritma berhasil menemukan solusi, akan menyajikan solusi dalam bentuk animasi langkah demi langkah pengisian puzzle tersebut.

```
Solution Founded in 301 tries in 0.0511627197265625 seconds!
Press enter to replay the solution, and exit to quit

Step 1 : Put Block 3 Rotated Once to (1, 3)

Board Copy matrix:
[-] [-] [B] [B]
[-] [-] [B] [Y]
[-]         [-]


Step 2 : Put Block 1 Rotated Once to (1, 4)

Board Copy matrix:
[-] [-] [B] [G]
[-] [-] [B] [G]
[-]         [Y]


Step 3 : Put Block 4 Not Flipped to (1, 1)

Board Copy matrix:
[B] [B] [B] [G]
[B] [B] [B] [G]
[-]         [Y]


Step 4 : Put Block 2 Rotated Once to (1, 1)

Board Copy matrix:
[G] [B] [B] [G]
[G] [B] [B] [G]
[Y]         [Y]
```

Gambar 3.7 Contoh Solusi
(Sumber : Dokumentasi Priabdi)

IV. Analisis dan Pembahasan

Algortima pencarian A* dianggap sudah sangat baik dalam melakukan pencarian jalan, namun pada worst case tetap saja sama seperti pencarian BFS dan DFS, dikarenakan akan mencoba semua kemungkinan, sehingga kompleksitasnya pencarian ialah O(n^n) pada kasus terburuk sedangkan untuk tiap node perlu dilakukan pengkopian matriks yang memakan O(max(width, height)^2), sehingga kasus terbaruk totalnya ialah O(n^n * max(width, height)^2), dimana n adalah jumlah balok yang memungkinkan untuk diletakkan. Namun pada kasus rata-rata, sangat bergantung pada fungsi heuristik, dan susah untuk dibuktikan secara asimtotik, namun dapat dipastikan bahwa A* jauh lebih optimal dari pencarian BFS maupun DFS.

Oleh karena itu dilakukan perbandingan antara algoritma BFS dan A* dengan level yang samasebagai berikut

Pada 3 kasus berikut, A* berhasil menyelesaikan dengan sejumlah :

```
Solving...

Solution Founded in 301 tries in 0.049803733825683594 seconds!
```

```
Solving...

Solution Founded in 9 tries in 0.004002809524536133 seconds!
```

```
Solving...

Solution Founded in 33 tries in 0.008010625839233398 seconds!
```

Sedangkan pada 3 kasus yang sama namun menggunakan pendekatan BFS, solusi dapat ditemukan dengan percobaan sejumlah :

```
Solving...

Solution Founded in 141 tries in 0.029497385025024414 seconds!
```

```
Solving...

Solution Founded in 99 tries in 0.03988814353942871 seconds!
```

```
Solving...

Solution Founded in 24 tries in 0.007014036178588867 seconds!
```

Dapat disimpulkan bahwa A* lebih cepat dari BFS.

V. Kesimpulan dan Saran

Algoritma pencarian A* dapat disimpulkan menjadi salah satu penyelesaian puzzle ini yang jauh lebih efektif dari pendekatan BFS maupun DFS. Optimisasi yang masih dapat dilakukan ialah dengan menambahkan catatan node yang telah dikunjungi, namun ada masalah ialah tiap node adalah matriks dan tidak bisa disimpan di dalam set untuk pencatatan node yang telah dikunjungi.

VI. Daftar Pustaka

[1] L. Turner, “Variants of the Shortest Path Problem,” Jan. 2011, Available: https://d-nb.info/1026787807/34. [Accessed: Jan. 10, 2025]

[2] A. Candra, M. A. Budiman, and K. Hartanto, “Dijkstra’s and A-Star in Finding the Shortest Path: a Tutorial,” *IEEE Xplore*, Jul. 01, 2020. doi: https://doi.org/10.1109/DATABIA50434.2020.9190342. Available: https://ieeexplore.ieee.org/document/9190342

[3] D. Rachmawati and L. Gustin, “Analysis of Dijkstra’s Algorithm and A* Algorithm in Shortest Path Problem,” *Journal of Physics: Conference Series*, vol. 1566, p. 012061, Jun. 2020, doi: https://doi.org/10.1088/1742-6596/1566/1/012061

[4] A. Muhamad and Adiputra, “Penerapan Algoritma BFS dan DFS untuk Penjadwalan Rencana Studi.” Available: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2017-2018/Makalah/Makalah-IF2211-2018-040.pdf

[5] N. Sugianti, A. Mardhiyah, and N. R. Fadlilah, “Komparasi Kinerja Algoritma BFS, Dijkstra, Greedy BFS, dan A* dalam Melakukan Pathfinding,” Jurnal Ilmiah

JISKa, vol. 5, no. 3, pp. 194-205, Nov. 2020. Available: https://ejournal.uin-suka.ac.id/saintek/JISKA/article/download/53-07/1786/6251

[6] D. S. Piskorskii, F. K. Abdullin, and A. R. Nikolaeva, "Optimization of A-Star Search Algorithm," *Bulletin of the South Ural State University. Ser. Computer Technologies, Automatic Control & Radioelectronics*, vol. 20, no. 1, pp. 154–160, 2020, doi: https://doi.org/10.14529/ctcr200115

[7] "numpy.ndarray — NumPy v2.1 Manual," *Numpy.org*, 2024. Available: https://numpy.org/doc/2.1/reference/generated/numpy.ndarray.html. [Accessed: Jan. 10, 2025]